

Level-1 Problem 1:

Scenario

A financial application needs to process multiple reports simultaneously to reduce waiting time. Instead of executing tasks sequentially, the system should run them concurrently using **C# Tasks** so that reports are generated faster.

Requirements

1. Create three methods:
 - a. GenerateSalesReport()
 - b. GenerateInventoryReport()
 - c. GenerateCustomerReport()
2. Each method should simulate processing time using Thread.Sleep() or Task.Delay().
3. Execute all three operations **concurrently using Task**.
4. Display a message when each report starts and when it finishes.
5. Display a final message once all reports are completed.

Technical Constraints

- Use **Task** class from **System.Threading.Tasks**.
- Use **Task.Run()** to execute methods.
- Use Task.WaitAll() or await **Task.WhenAll()** to wait for completion.
- The program must run in a **Console Application**.

Expectations

The program should start multiple report-generation tasks simultaneously and display completion messages for each report along with a final message once all tasks are completed.

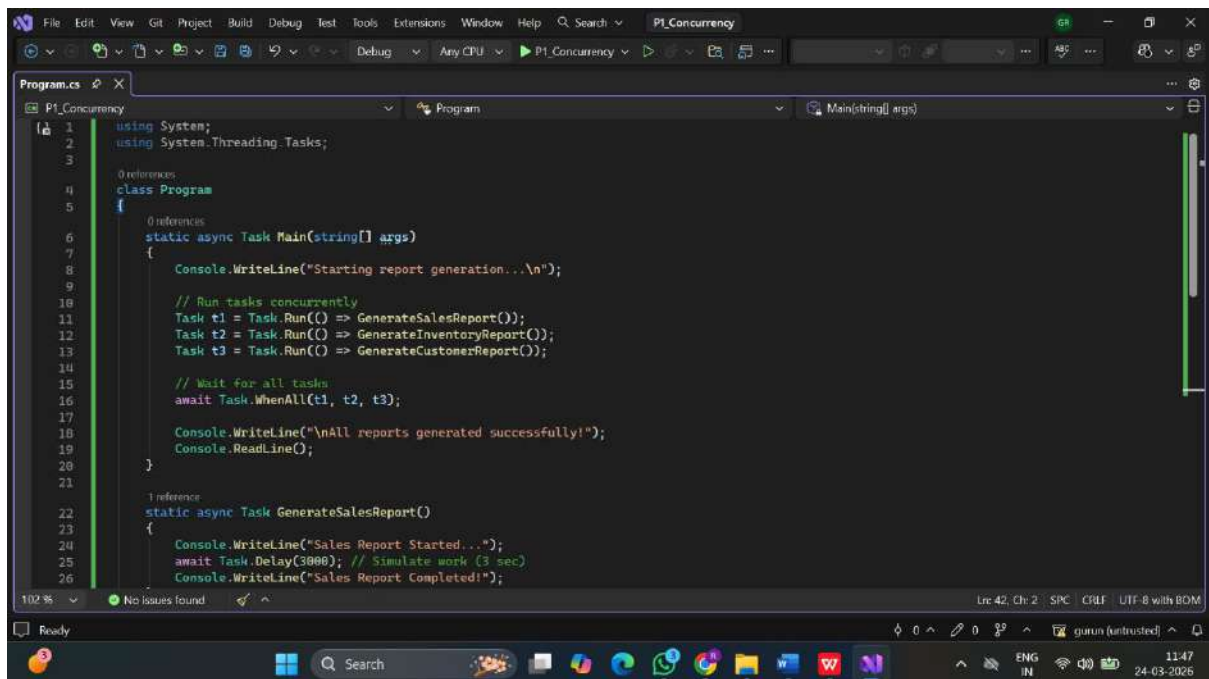
Learning Outcome

Students will learn:

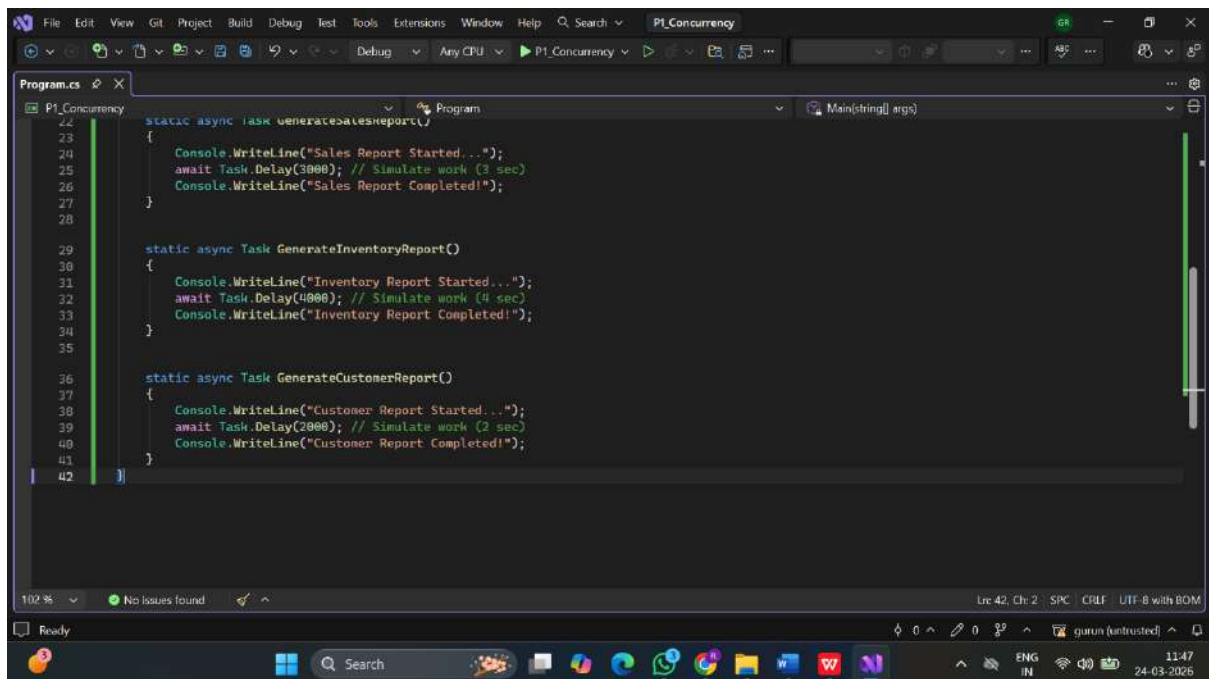
- How to create and run **Tasks in C#**
- How to execute **multiple operations concurrently**

How to **wait for multiple tasks to complete**

Code Screenshot:

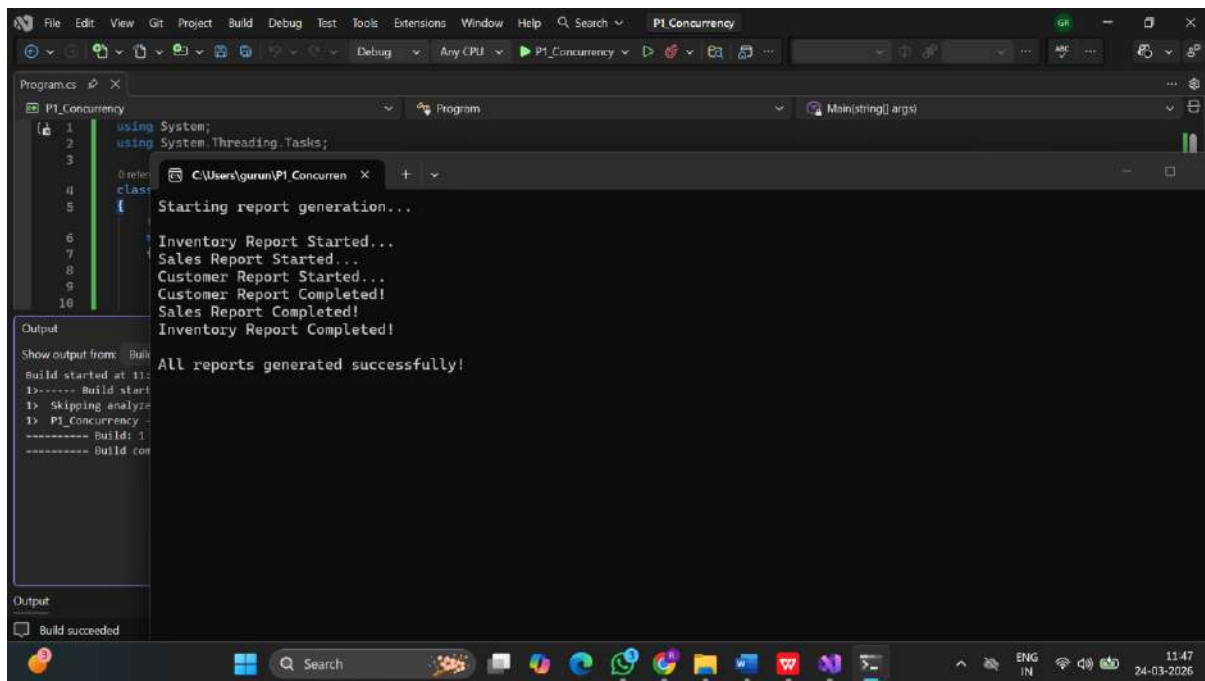


```
1 using System;
2 using System.Threading.Tasks;
3
4 class Program
5 {
6     // References
7     static async Task Main(string[] args)
8     {
9         Console.WriteLine("Starting report generation...\n");
10
11         // Run tasks concurrently
12         Task t1 = Task.Run(() => GenerateSalesReport());
13         Task t2 = Task.Run(() => GenerateInventoryReport());
14         Task t3 = Task.Run(() => GenerateCustomerReport());
15
16         // Wait for all tasks
17         await Task.WhenAll(t1, t2, t3);
18
19         Console.WriteLine("\nAll reports generated successfully!");
20         Console.ReadLine();
21     }
22
23     // 1 reference
24     static async Task GenerateSalesReport()
25     {
26         Console.WriteLine("Sales Report Started...");
27         await Task.Delay(3000); // Simulate work (3 sec)
28         Console.WriteLine("Sales Report Completed!");
29     }
30 }
```



```
29
30 static async Task GenerateInventoryReport()
31 {
32     Console.WriteLine("Inventory Report Started...");
33     await Task.Delay(4000); // Simulate work (4 sec)
34     Console.WriteLine("Inventory Report Completed!");
35 }
36
37 static async Task GenerateCustomerReport()
38 {
39     Console.WriteLine("Customer Report Started...");
40     await Task.Delay(2000); // Simulate work (2 sec)
41     Console.WriteLine("Customer Report Completed!");
42 }
```

Output:



```
using System;
using System.Threading.Tasks;

class Program
{
    static void Main(string[] args)
    {
        Starting report generation...
        Inventory Report Started...
        Sales Report Started...
        Customer Report Started...
        Customer Report Completed!
        Sales Report Completed!
        Inventory Report Completed!
        All reports generated successfully!
    }
}
```

Level-1 Problem 2: Asynchronous File Logger

Scenario:

An application writes logs to a file whenever an event occurs. Writing logs synchronously can slow down the application. Asynchronous file writing improves performance.

Requirements:

- Create an asynchronous method `WriteLogAsync(string message)`.
- The method should simulate file writing using `Task.Delay()`.
- Call this method multiple times to simulate logging different events.

Technical Constraints:

- Use `async` and `await` keywords.
- Use `Task.Delay()` to simulate file I/O.
- Use a console application.

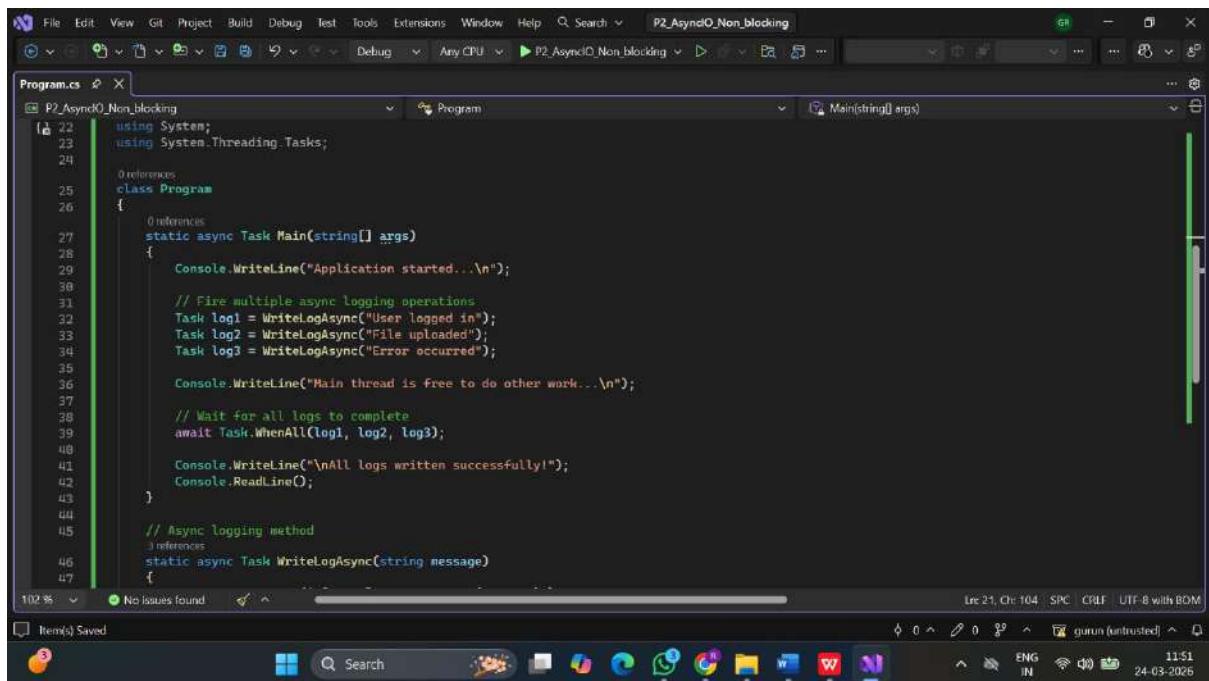
Expectations:

- Logs should be written asynchronously.
- The main thread should remain responsive while logging operations occur.

Learning Outcome:

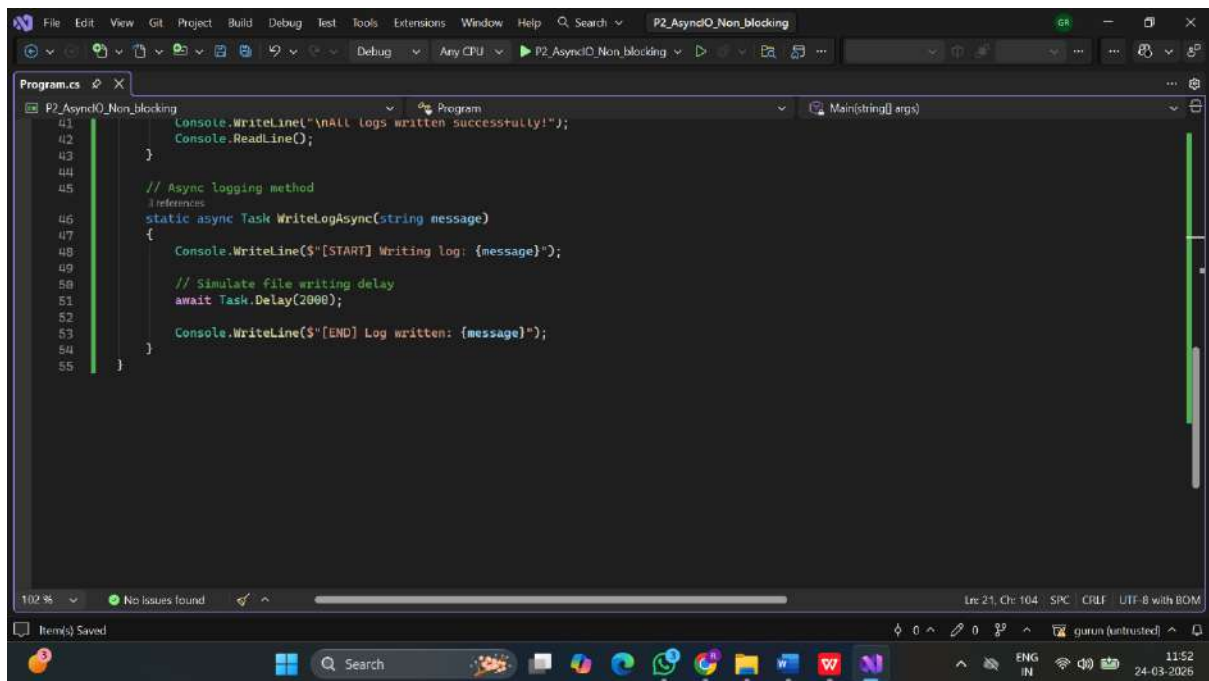
Students will learn how asynchronous operations improve performance when dealing with I/O operations.

Code Screenshots:



```
Program.cs
P2_AsyncIO_Non_blocking
Program
Main(string[] args)

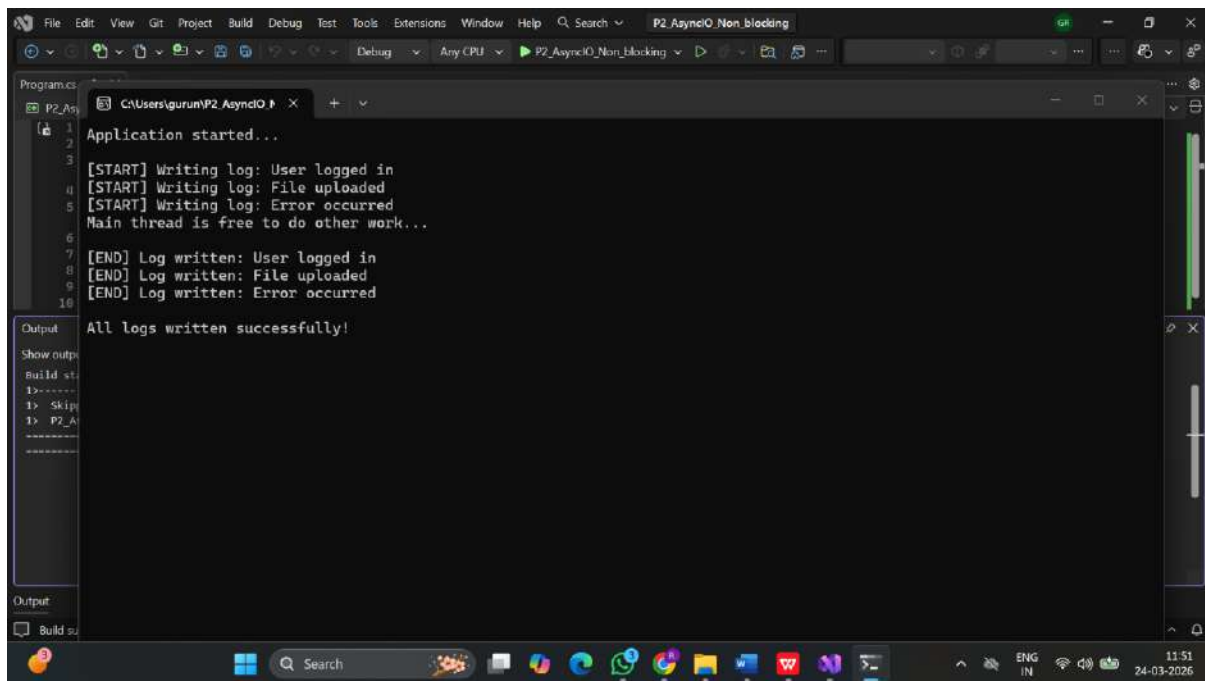
22 using System;
23 using System.Threading.Tasks;
24
25 0 references
26 class Program
27 {
28     0 references
29     static async Task Main(string[] args)
30     {
31         Console.WriteLine("Application started...\n");
32
33         // Fire multiple async logging operations
34         Task log1 = WriteLogAsync("User logged in");
35         Task log2 = WriteLogAsync("File uploaded");
36         Task log3 = WriteLogAsync("Error occurred");
37
38         Console.WriteLine("Main thread is free to do other work...\n");
39
40         // Wait for all logs to complete
41         await Task.WhenAll(log1, log2, log3);
42
43         Console.WriteLine("\nAll logs written successfully!");
44         Console.ReadLine();
45     }
46
47     // Async logging method
48     0 references
49     static async Task WriteLogAsync(string message)
50     {
51     }
```



```
Program.cs
P2_AsyncIO_Non_blocking
Program
Main(string[] args)

41 Console.WriteLine("\nAll logs written successfully!");
42 Console.ReadLine();
43 }
44
45 // Async logging method
46 0 references
47 static async Task WriteLogAsync(string message)
48 {
49     Console.WriteLine($"[START] Writing log: {message}");
50
51     // Simulate file writing delay
52     await Task.Delay(2000);
53
54     Console.WriteLine($"[END] Log written: {message}");
55 }
```

Output:



```
Program.cs
1 Application started...
2
3 [START] Writing log: User logged in
4 [START] Writing log: File uploaded
5 [START] Writing log: Error occurred
6 Main thread is free to do other work...
7
8 [END] Log written: User logged in
9 [END] Log written: File uploaded
10 [END] Log written: Error occurred
11
12 All logs written successfully!
```

Level-2 Problem 1: Asynchronous Order Processing System

Scenario:

An e-commerce system processes customer orders. Each order requires multiple steps such as payment verification, inventory check, and order confirmation. These steps involve delays and should be handled asynchronously.

Requirements:

- Create asynchronous methods:
 - VerifyPaymentAsync()
 - CheckInventoryAsync()
 - ConfirmOrderAsync()
- Simulate processing delays using Task.Delay().
- Execute steps asynchronously while maintaining the logical order of operations.

Technical Constraints:

- Use async and await.
- Use Task-based asynchronous methods.
- Implement using a console application.

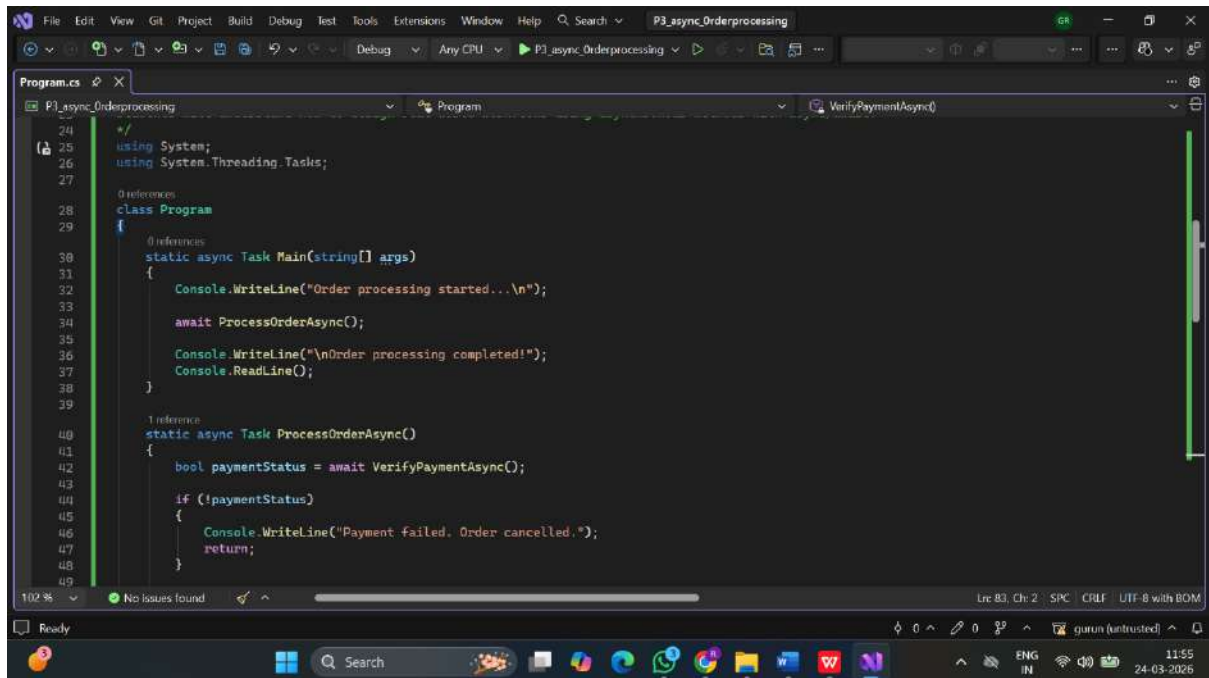
Expectations:

- Each processing step should run asynchronously.
- The program should display messages indicating the progress of order processing.

Learning Outcome:

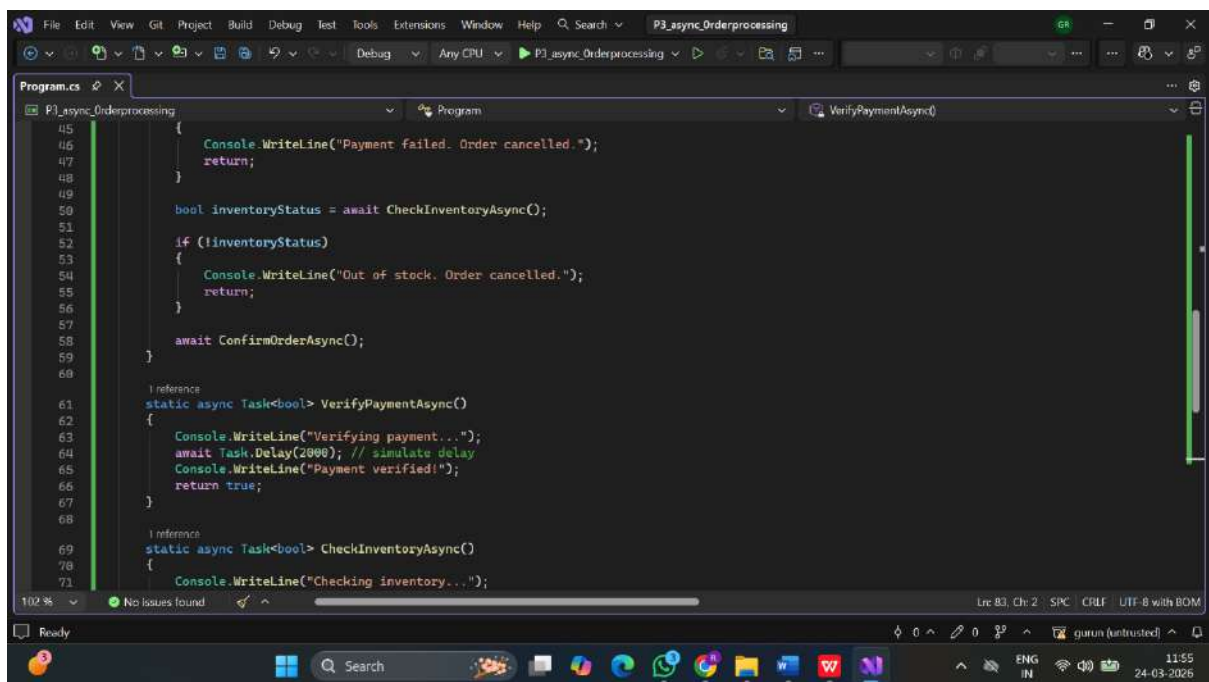
Students will understand how to design real-world workflows using asynchronous methods with async/await.

Code Screenshot:



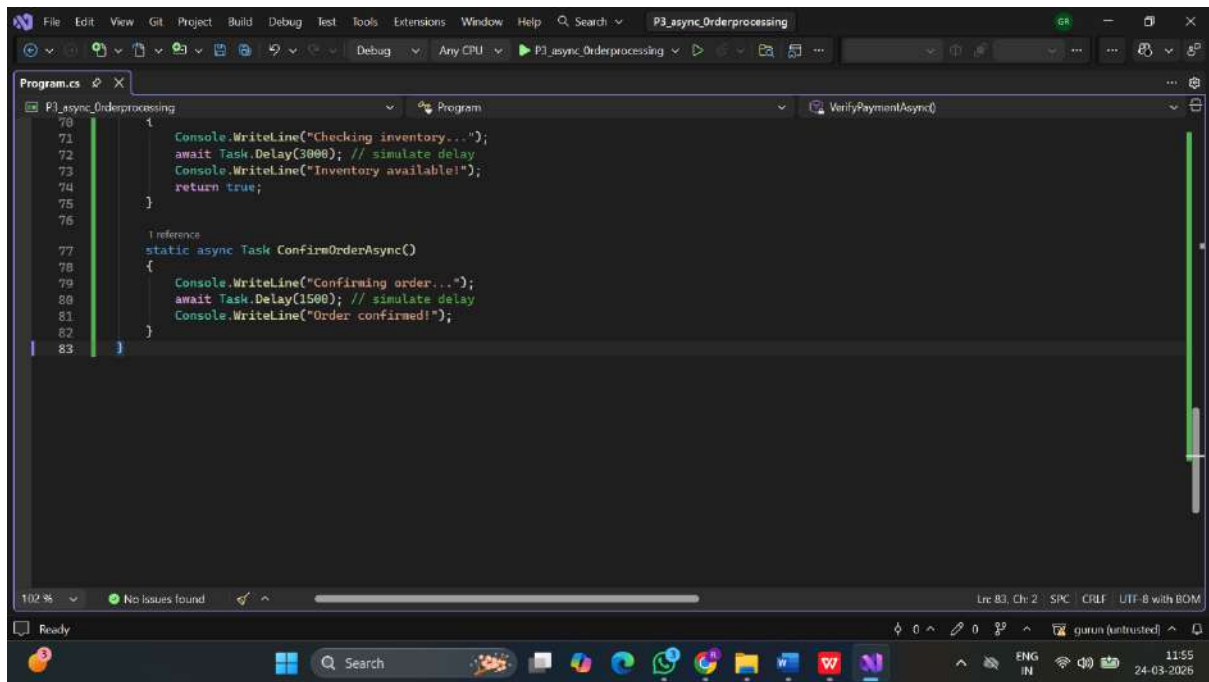
This screenshot shows the first part of the `Program.cs` file in Visual Studio. The code defines a `Program` class with a `Main` method and a `ProcessOrderAsync` method. The `Main` method starts by writing "Order processing started...", then awaits `ProcessOrderAsync`, and finally writes "Order processing completed!" and reads a line from the console. The `ProcessOrderAsync` method starts by awaiting `VerifyPaymentAsync`. If the payment is not successful, it writes "Payment failed. Order cancelled." and returns. The `VerifyPaymentAsync` method is shown in the right-hand pane.

```
24 //
25 using System;
26 using System.Threading.Tasks;
27
28 0 references
29 class Program
30 {
31     0 references
32     static async Task Main(string[] args)
33     {
34         Console.WriteLine("Order processing started...\n");
35
36         await ProcessOrderAsync();
37
38         Console.WriteLine("\nOrder processing completed!");
39         Console.ReadLine();
40     }
41
42     1 reference
43     static async Task ProcessOrderAsync()
44     {
45         bool paymentStatus = await VerifyPaymentAsync();
46
47         if (!paymentStatus)
48         {
49             Console.WriteLine("Payment failed. Order cancelled.");
50             return;
51         }
52     }
53 }
```



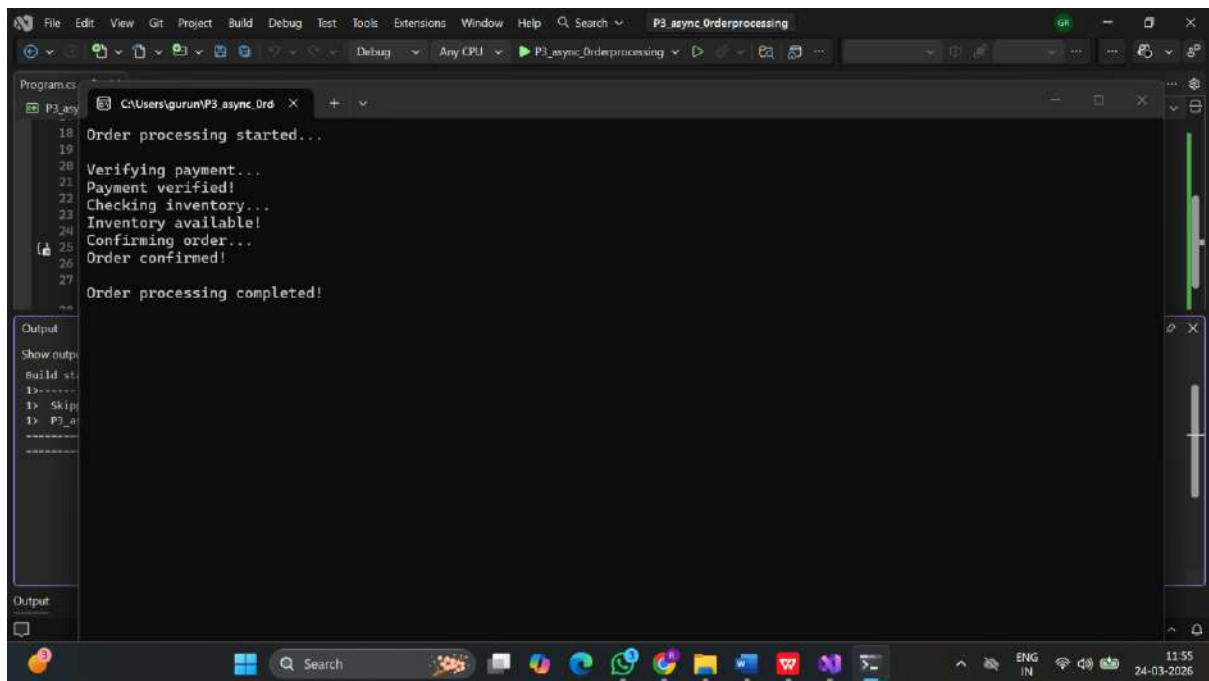
This screenshot shows the second part of the `Program.cs` file in Visual Studio. The code continues the `ProcessOrderAsync` method by awaiting `CheckInventoryAsync`. If the inventory is not available, it writes "Out of stock. Order cancelled." and returns. Then it awaits `ConfirmOrderAsync`. The `VerifyPaymentAsync` method is shown in the right-hand pane, which writes "Verifying payment...", delays for 2000ms, writes "Payment verified!", and returns true. The `CheckInventoryAsync` method is shown in the right-hand pane, which writes "Checking inventory...".

```
45 {
46     Console.WriteLine("Payment failed. Order cancelled.");
47     return;
48 }
49
50 bool inventoryStatus = await CheckInventoryAsync();
51
52 if (!inventoryStatus)
53 {
54     Console.WriteLine("Out of stock. Order cancelled.");
55     return;
56 }
57
58 await ConfirmOrderAsync();
59 }
60
61 1 reference
62 static async Task<bool> VerifyPaymentAsync()
63 {
64     Console.WriteLine("Verifying payment...");
65     await Task.Delay(2000); // simulate delay
66     Console.WriteLine("Payment verified!");
67     return true;
68 }
69
70 1 reference
71 static async Task<bool> CheckInventoryAsync()
72 {
73     Console.WriteLine("Checking inventory...");
74 }
```



```
Program.cs
P3_async_Orderprocessing
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71 Console.WriteLine("Checking inventory...");
72 await Task.Delay(3000); // simulate delay
73 Console.WriteLine("Inventory available!");
74 return true;
75 }
76
77 1 reference
78 static async Task ConfirmOrderAsync()
79 {
80 Console.WriteLine("Confirming order...");
81 await Task.Delay(1500); // simulate delay
82 Console.WriteLine("Order confirmed!");
83 }
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Output:



```
Program.cs
P3_async_Orderprocessing
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18 Order processing started...
19
20
21 Verifying payment...
22 Payment verified!
23 Checking inventory...
24 Inventory available!
25 Confirming order...
26 Order confirmed!
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Level-2 Problem 2: Debugging Incorrect Discount Calculation

Scenario

A retail application calculates the final price of products after applying a discount. Recently, users reported that the final price shown by the application is incorrect. The development team needs to **debug the application to identify where the incorrect calculation is happening**.

Requirements

- Create a console application that calculates the final product price.
- The program should accept:
 - Product Name
 - Product Price
 - Discount Percentage
- The final price should be calculated using the formula:

$$\text{FinalPrice} = \text{Price} - (\text{Price} \times \text{Discount} / 100)$$

- Use **debugging tools** to verify that the calculation is correct.
- Place **breakpoints** and inspect variable values during execution.

Technical Constraints

- Use **Visual Studio Debugging Tools**.
- Use **breakpoints, step over, step into, and watch window**.
- Implement the solution using a **.NET console application**.

Expectations

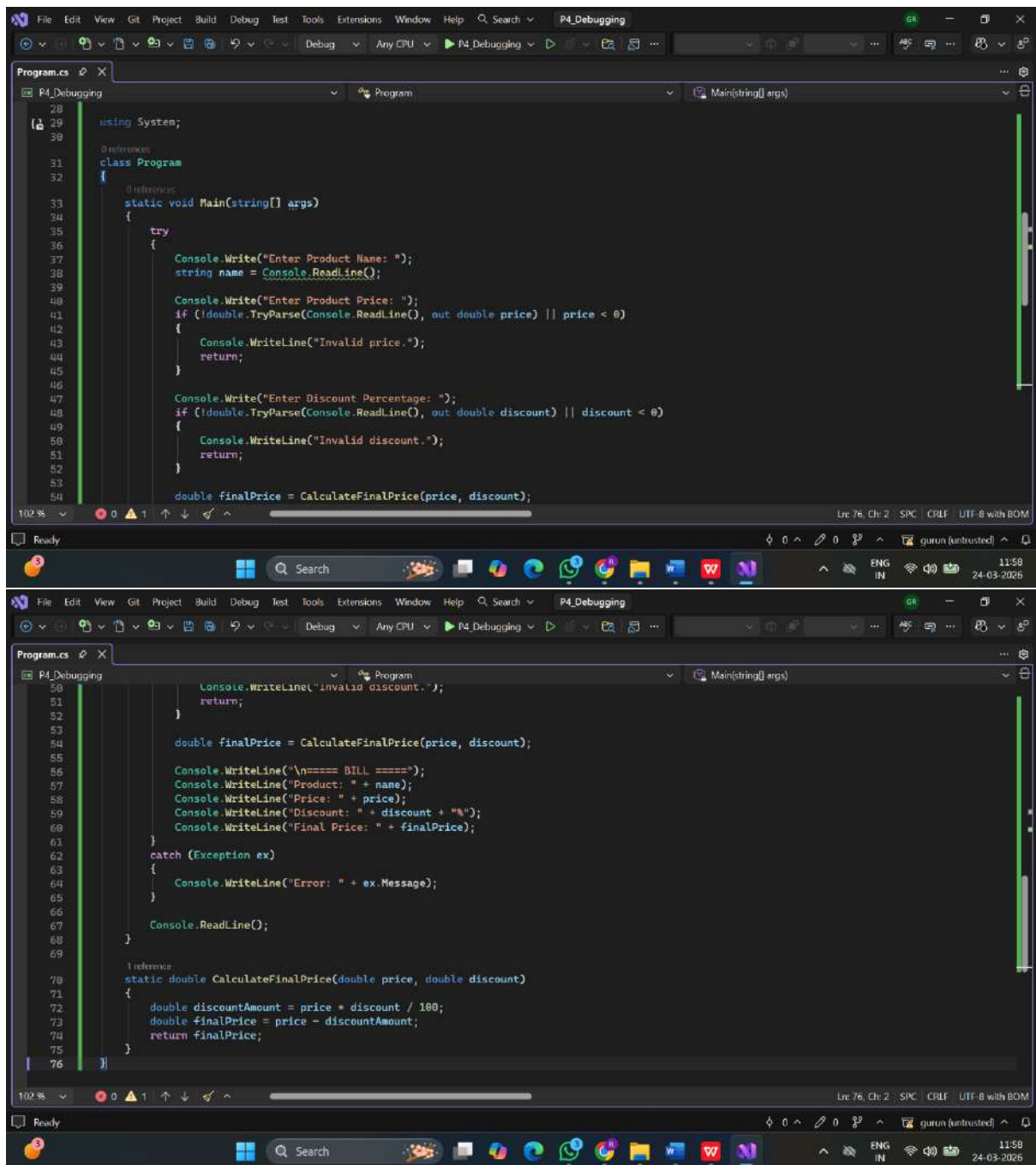
- Students should run the program in **debug mode**.
- They should track variable values and confirm that the discount calculation is correct.
- If incorrect results appear, students should identify the faulty logic.

Learning Outcome

Students will learn how to:

- Use **breakpoints effectively**.
- Inspect **variable values during program execution**.
- Identify logical errors using **debugging techniques**.

Code Screenshot:



The image displays two screenshots of a Visual Studio code editor window, showing a C# program named `Program.cs`. The editor is in the `Debug` mode, and the file is named `P4_Debugging`.

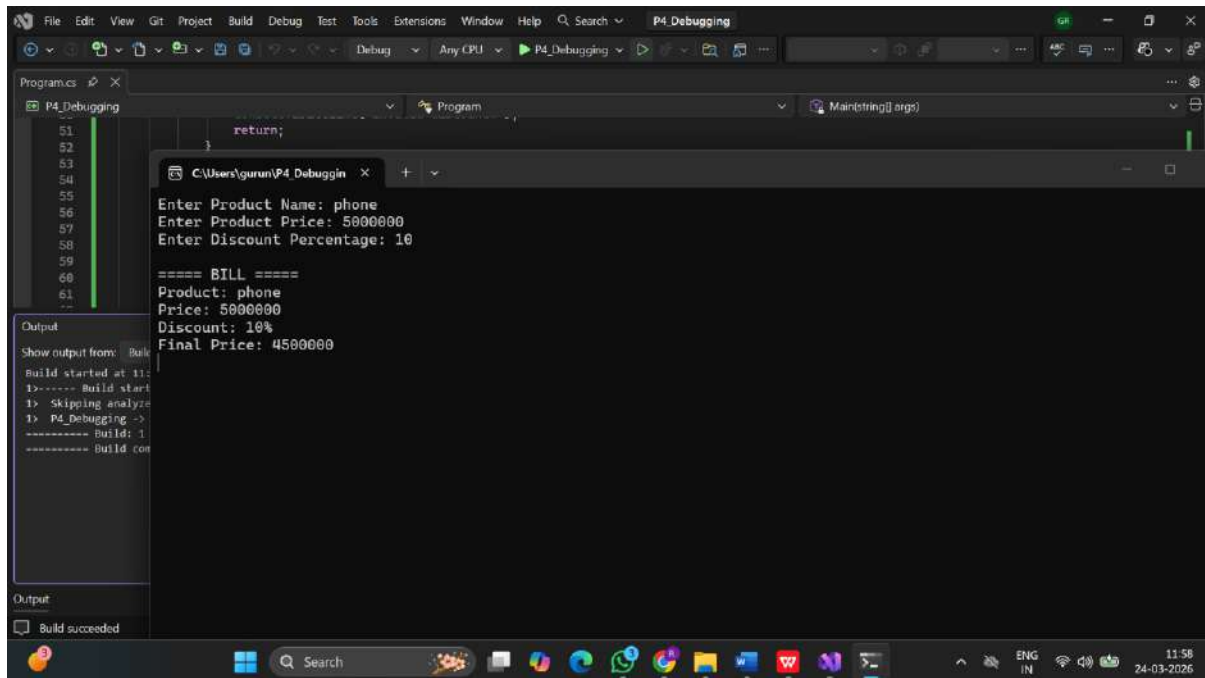
The first screenshot shows the `Main` method, which prompts the user to enter the product name, price, and discount percentage. It includes input validation for the price and discount, and calls the `CalculateFinalPrice` method.

```
28 using System;
29
30 namespace P4_Debugging
31 {
32     class Program
33     {
34         static void Main(string[] args)
35         {
36             try
37             {
38                 Console.WriteLine("Enter Product Name: ");
39                 string name = Console.ReadLine();
40
41                 Console.WriteLine("Enter Product Price: ");
42                 if (!double.TryParse(Console.ReadLine(), out double price) || price < 0)
43                 {
44                     Console.WriteLine("Invalid price.");
45                     return;
46                 }
47
48                 Console.WriteLine("Enter Discount Percentage: ");
49                 if (!double.TryParse(Console.ReadLine(), out double discount) || discount < 0)
50                 {
51                     Console.WriteLine("Invalid discount.");
52                     return;
53                 }
54
55                 double finalPrice = CalculateFinalPrice(price, discount);
56
57                 Console.WriteLine("\n==== BILL =====");
58                 Console.WriteLine("Product: " + name);
59                 Console.WriteLine("Price: " + price);
60                 Console.WriteLine("Discount: " + discount + "%");
61                 Console.WriteLine("Final Price: " + finalPrice);
62
63                 catch (Exception ex)
64                 {
65                     Console.WriteLine("Error: " + ex.Message);
66                 }
67
68                 Console.ReadLine();
69             }
70
71             static double CalculateFinalPrice(double price, double discount)
72             {
73                 double discountAmount = price * discount / 100;
74                 double finalPrice = price - discountAmount;
75                 return finalPrice;
76             }
77         }
78     }
79 }
```

The second screenshot shows the `CalculateFinalPrice` method, which calculates the final price by subtracting the discount amount from the original price.

```
70 static double CalculateFinalPrice(double price, double discount)
71 {
72     double discountAmount = price * discount / 100;
73     double finalPrice = price - discountAmount;
74     return finalPrice;
75 }
76 }
```

Output:



```
return;
```

```
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
--
```

```
Enter Product Name: phone  
Enter Product Price: 5000000  
Enter Discount Percentage: 10  
  
===== BILL =====  
Product: phone  
Price: 5000000  
Discount: 10%  
Final Price: 4500000
```

```
Build started at 11:58:11  
1>----- Build start  
1> Skipping analyze  
1> P4_Debugging ->  
----- Build: 1  
----- Build complete
```

```
Build succeeded
```

Level-2 Problem 3: Application Tracing for Order Processing

Scenario

An e-commerce application processes customer orders. Sometimes the order processing fails, but developers are unable to identify where the failure occurs. The team decides to implement **Tracing** to monitor the execution flow and diagnose the issue.

Requirements

- Create a console application that simulates **order processing**.
- The order processing should include the following steps:
 1. Validate Order
 2. Process Payment
 3. Update Inventory
 4. Generate Invoice
- Use **Trace class** to log messages at each step of the process.
- Display trace messages showing the execution flow.

Technical Constraints

- Use **System.Diagnostics.Trace**.
- Write trace messages using:
 - `Trace.WriteLine()`

- `Trace.TraceInformation()`
- Configure a **TextWriterTraceListener** to store trace logs in a file.
- Implement the solution using **.NET console application**.

Expectations

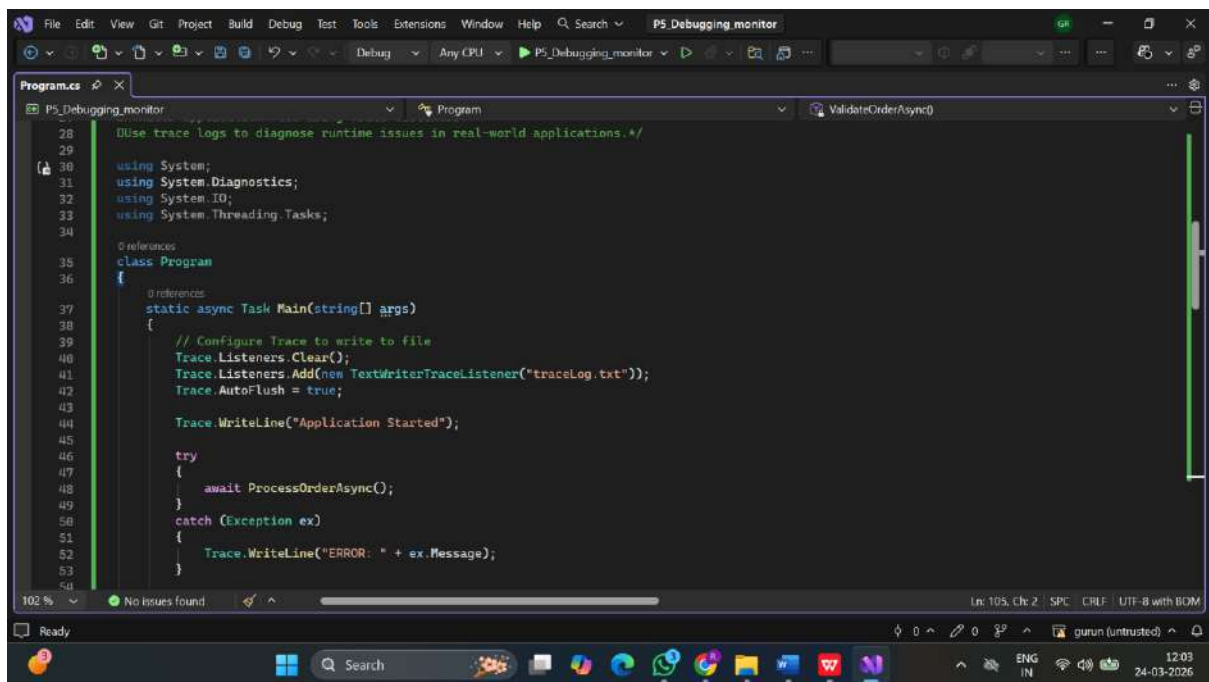
- The application should log messages for each stage of order processing.
- Trace logs should help developers identify where failures occur.
- The trace output should be stored in a **log file**.

Learning Outcome

Students will learn how to:

- Implement **application tracing** using `System.Diagnostics`.
- Monitor application flow using **Trace listeners**.
- Use trace logs to **diagnose runtime issues in real-world applications**.

Code Screenshot:



The screenshot shows the Visual Studio IDE with a C# console application named 'P5_Debugging_monitor'. The code is as follows:

```
28 // Use trace logs to diagnose runtime issues in real-world applications.*/
29
30 using System;
31 using System.Diagnostics;
32 using System.IO;
33 using System.Threading.Tasks;
34
35 namespace P5_Debugging_monitor
36 {
37     class Program
38     {
39         // Configure Trace to write to file
40         static async Task Main(string[] args)
41         {
42             Trace.Listeners.Clear();
43             Trace.Listeners.Add(new TextWriterTraceListener("traceLog.txt"));
44             Trace.AutoFlush = true;
45
46             Trace.WriteLine("Application Started");
47
48             try
49             {
50                 await ProcessOrderAsync();
51             }
52             catch (Exception ex)
53             {
54                 Trace.WriteLine("ERROR: " + ex.Message);
55             }
56         }
57     }
58 }
```

The code demonstrates how to configure `Trace` to write logs to a file named `traceLog.txt`. It includes a `try-catch` block to handle exceptions and log error messages.

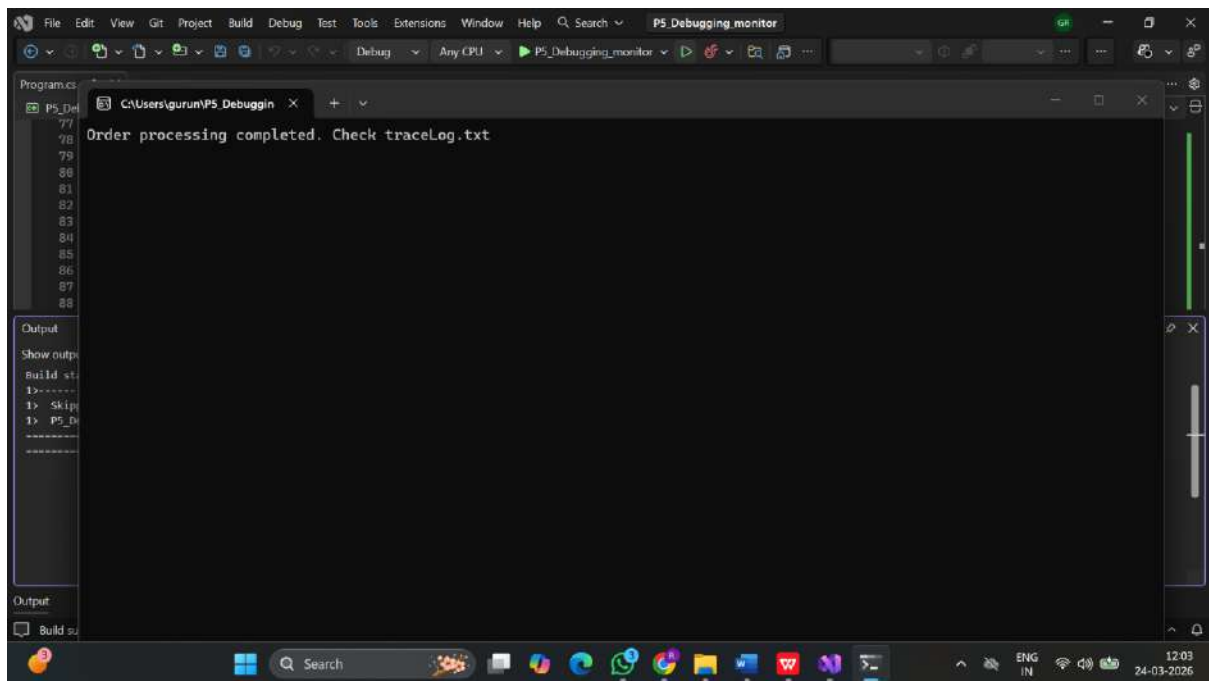
This screenshot shows the first part of the `Program.cs` file in Visual Studio. The code includes a `main` method that calls `ValidateOrderAsync()`, followed by `ProcessOrderAsync()` which awaits `ValidateOrderAsync()`, `ProcessPaymentAsync()`, `UpdateInventoryAsync()`, and `GenerateInvoiceAsync()`. The `ValidateOrderAsync()` method is shown with a `Trace` call and a `Task.Delay(1000)`. The `ProcessPaymentAsync()` method is partially visible at the bottom.

```
53 }
54
55 Trace.WriteLine("Application Ended");
56
57 Console.WriteLine("Order processing completed. Check traceLog.txt");
58 Console.ReadLine();
59 }
60
61 static async Task ProcessOrderAsync()
62 {
63     await ValidateOrderAsync();
64     await ProcessPaymentAsync();
65     await UpdateInventoryAsync();
66     await GenerateInvoiceAsync();
67 }
68
69 static async Task ValidateOrderAsync()
70 {
71     Trace.TraceInformation("Step 1: Validating Order...");
72     await Task.Delay(1000);
73     Trace.WriteLine("Order validated successfully.");
74 }
75
76 // reference
77 static async Task ProcessPaymentAsync()
78 {
79     Trace.TraceInformation("Step 2: Processing Payment...");
```

This screenshot shows the second part of the `Program.cs` file. It continues the `ProcessPaymentAsync()` method with a `Task.Delay(1500)`, a comment about simulating failure, a `bool paymentSuccess` variable, and an `if` block that throws an exception if payment fails. It then shows the `UpdateInventoryAsync()` and `GenerateInvoiceAsync()` methods, each with a `Trace` call and a `Task.Delay`.

```
77     Trace.TraceInformation("Step 2: Processing Payment...");
78     await Task.Delay(1500);
79
80     // Simulate failure (for debugging demo)
81     bool paymentSuccess = true;
82
83     if (!paymentSuccess)
84     {
85         throw new Exception("Payment failed!");
86     }
87
88     Trace.WriteLine("Payment processed successfully.");
89 }
90
91 // reference
92 static async Task UpdateInventoryAsync()
93 {
94     Trace.TraceInformation("Step 3: Updating Inventory...");
95     await Task.Delay(1200);
96     Trace.WriteLine("Inventory updated successfully.");
97 }
98
99 // reference
100 static async Task GenerateInvoiceAsync()
101 {
102     Trace.TraceInformation("Step 4: Generating Invoice...");
103     await Task.Delay(800);
104     Trace.WriteLine("Invoice generated successfully.");
105 }
```

Output:



The screenshot shows the Visual Studio Code interface with the 'P5_Debugging_monitor' window open. The window displays the text 'Order processing completed. Check traceLog.txt'. The left sidebar shows the 'Program.cs' file with line numbers 77 through 88. The bottom status bar indicates the language is 'ENG IN' and the date is '24-03-2026'.

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P5_Debugging_monitor
Debug Any CPU P5_Debugging_monitor
Program.cs
P5_Debugging_monitor
77
78
79
80
81
82
83
84
85
86
87
88
Order processing completed. Check traceLog.txt
Output
Show output
Build st.
1>-----
1> Skip
1> P5_De
-----
Output
Build su
12:03
24-03-2026
```